

BlockVote: Master MCA Project Viva Guide & IEEE Research Paper Analysis

Candidate / Lead Author: Deven Wagh (Department of Computer Applications, Pillai HOC College of Engineering & Technology, Rasayani)

Faculty Guide: Dr. Abhijeet More | **Co-Authors:** Kalpit Mhatre, Rohit Zagade

Paper Title: BlockVote: A Gasless, Coercion-Mitigating Decentralized E-Voting System Combining Sensor-Driven Biometric Liveness, UUPS Multi-Signature Governance, and On-Chain Keccak-256 Nullifiers

21,609

LINES OF CODE (182 FILES)

190 / 190

PASSING TESTS (30 ATTACKS)

89,946

GAS PER BALLOT (V3)

114

LIVE SEPOLIA TXS ANALYZED

RESEARCH PAPER ANALYSIS: BlockVote_IEEE_merged.pdf

This section provides a thorough, part-by-part explanation of the research paper written for this project (*BlockVote: A Gasless, Coercion-Mitigating Decentralized E-Voting System Combining Sensor-Driven Biometric Liveness, UUPS Multi-Signature Governance, and On-Chain Keccak-256 Nullifiers* by Deven Wagh, Kalpit Mhatre, Rohit Zagade, and Dr. Abhijeet More, Pillai HOC College of Engineering & Technology).

1. Paper Abstract & Core Problem Formulation

Motivation: Conventional paper ballots suffer from physical ballot-box snatching, high logistical overheads, manual counting delays, and human bias. Conversely, existing electronic voting machines (EVMs) require physical presence, while traditional web-based e-voting portals rely on centralized databases (e.g. MySQL/PostgreSQL) where database administrators (DBAs) or compromised servers can secretly modify vote tallies without an audit trail.

The Core Dilemma: While blockchain offers decentralized immutability and verifiable computation, standard blockchain voting requires voters to install browser extensions (MetaMask), hold private keys, and pay gas fees in volatile cryptocurrency (ETH/MATIC). This forms an impossible barrier for non-technical citizens. Furthermore, public blockchain transactions expose wallet addresses and choice parameters, violating ballot secrecy.

BlockVote's Hybrid Breakthrough: BlockVote bridges this divide through an upgradeable Ethereum smart contract (`VotingV3` , UUPS proxy with 2-of-3 guardian multi-signature governance) that keys voters by salted Keccak-256 nullifiers and emits salted ballot commitments. The voter has zero blockchain overhead: a server-side gasless relay with a serialized promise queue submits and sponsors all transactions. Voters authenticate via email OTP and AWS Rekognition face liveness, while the native Android app enforces a 240° sensor-driven rotation scan. Coercion is mitigated via bounded re-voting (first vote + one change) and a final physical polling-station override.

2. Section I: Introduction & The 6 Scientific Contributions

The paper explicitly outlines six technical contributions distinguishing BlockVote from previous literature:

- Gasless Relay with Serialized Nonce Queue:** A platform-sponsored relay wallet executes `castVoteRelayedV3()` , eliminating voter cryptocurrency wallets and gas fees, while a promise queue and `NonceManager` eliminate EVM nonce collision during concurrent voting bursts.
- Salted Keccak-256 Nullifier (Zero-PII On-Chain):** Voters are identified on-chain exclusively via `keccak256(clean(email) || ":" || K_id)` , decoupling personal identifiable information (PII) from on-chain state while guaranteeing per-election uniqueness.
- Sensor-Driven Biometric Liveness Workflow:** Android motion sensing (240° yaw tracking via `SensorManager`), ML Kit real-time face tracking (rejecting multi-face frames), and AWS Rekognition server-side validation (confidence $\geq 90\%$, open eyes, no glasses).
- Blinded Ballots & Bounded Multi-Channel Re-Voting:** On-chain events emit only salted hashes `keccak256(electionId || candidateId || nullifier || salt)` . Voters can cast 2 app votes (first vote + 1 change of mind); a physical polling-station vote is final and overrides remote ballots.

5. **UUPS Upgradeability with 2-of-3 Multi-Signature Governance:** Contract upgrades follow EIP-1822 Universal Upgradeable Proxy Standard requiring on-chain approvals from at least two out of three independent guardian cryptographic key holders.
6. **Comprehensive Empirical Evaluation:** 190 automated unit and security tests, gas consumption cross-validation between Hardhat local RPC and 114 live Ethereum Sepolia transactions, and Indian residential production latency profiling.

3. Section II: Literature Survey & Qualitative Comparison

The paper surveys five generations of voting systems, highlighting trade-offs across 10 security and usability dimensions:

Evaluation Property	EVM + VVPAT	Estonian i-Voting	Helios	Voatz	Open Vote Network	BlockVote (This Work)
Remote Voting	No (Physical Booth)	Yes	Yes	Yes	Yes	Yes (Android App)
Supervised In-Person Channel	Yes	Yes	No	No	No	Yes (/station kiosk)
Re-Voting / Physical Override	No	Yes (Estonian model)	Partial	No	No	Yes (Bounded + Override)
Public Tamper-Evident Ledger	No	No	Yes (Bulletin Board)	Partial (Permissioned)	Yes (Ethereum)	Yes (Ethereum Sepolia)
Universally Verifiable Tally	Partial (Paper VVPAT)	No	Yes (Homomorphic)	No	Yes (Self-Tallying)	Yes (On-Chain Counters)
Ballot Secrecy vs. Public	Yes	Yes	Yes	Partial	Yes	Yes (Salted Hash)
Ballot Secrecy vs. Operator	Yes	Partial (Key Holders)	Partial (Trustees)	No	Yes	Relayer sees submission
Automated Liveness / Biometrics	No (Manual ID)	No	No	Yes (Face)	No	Yes (Sensor + AWS)
No Voter Wallet or Gas Fees	Yes	Yes	Yes	Yes	No (Voter pays)	Yes (Gasless Relayer)
Upgrade Governance	N/A	N/A	N/A	N/A	No (Immutable)	Yes (2-of-3 UUPS)

4. Section III: Methodology & Mathematical Formulations

BlockVote's operational mechanics are governed by formal mathematical specifications and algorithms:

A. Nullifier Derivation (Equation 1)

To record registration and prevent double voting without writing email addresses to the blockchain:

```
n = Keccak256( clean(email) || ":" || K_id )
```

Where `clean(email)` represents lowercase trimmed ASCII string, and `K_id` is a high-entropy 256-bit server secret (`SERVER_IDENTITY_SECRET`) protected by fail-closed environment checks. The contract stores `isRegisteredVoter[e][n]` and `voteChoice[n][e]`.

B. Blinded Ballot Event (Equation 2)

In legacy VotingV1, the contract emitted `VoteCast(electionId, candidateId)` which leaked candidate popularity and timing. VotingV3 emits:

```
VoteCastPrivate( electionId, Keccak256(electionId || candidateId || n || s), isRevote )
```

Where `s` is a cryptographically secure 32-byte pseudo-random salt generated by the backend relayer. Because `s` is 256 bits, dictionary pre-computation attacks by public node observers are mathematically impossible.

C. UUPS 2-of-3 Multi-Signature Upgrade Rule (Equation 3)

The contract authorization gate `_authorizeUpgrade(address newImplementation)` enforces:

```
UpgradeAccepted = (approvalCount >= 2) AND (executed == false)
```

Only three registered guardian hardware/wallet addresses can call `proposeUpgrade()` and `approveUpgrade()`. A single compromised key cannot alter election logic.

D. Atomic Vote-Slot Reservation (Algorithm 1)

To prevent race conditions where a voter attempts concurrent voting requests across multiple tabs or phones to bypass the 2-vote limit:

```
1: Filter F = { _id: voterId, stationVoteFinal: { $ne: true } }
2: If channel == 'app': F = F AND (votesCast < 2)
3: Update U = { $inc: { votesCast: 1 } }
4: If channel == 'station': U = U UNION { stationVoteFinal: true }
5: b = FindOneAndUpdate(F, U, return before)
6: If b == null: REJECT (VOTE_LIMIT_REACHED or STATION_VOTE_FINAL)
7: r = RelayCastVote(...)
8: If r fails: RESTORE voter document to state b (Slot rollback)
```

E. Codebase Size Metrics (Table II in Paper)

Component	Files	Lines of Code (LOC)
Solidity Smart Contracts (V1, V2, V3)	3	959
API Route Handlers (Next.js 15)	56	4,735
Web Pages and Components (JSX / React 19)	44	7,699
Server Libraries & Helpers (JS)	37	2,865
Android Native App (Kotlin 2.2 / Compose)	42	5,351
Total Codebase	182	21,609

5. Section IV: Experimental Results, Gas Benchmarks & Production Data

A. Automated Testing Verification (190 Passing Tests)

- **VotingV3 Smart Contract Suite:** 75 tests (Contract deployment, lifecycle, candidate tally decrement/increment, underflow guards, UUPS proxy upgrade authorization).
- **Adversarial Security Suite (A1 to A8):** 30 tests covering malicious actors:
 - *A1 Voter:* Unregistered vote rejected, zero nullifier rejected, direct non-relay call rejected, candidate ID out-of-range rejected.
 - *A2 Coercer:* Verifies candidate ID is absent from logs, legacy events disabled, coerced vote overwritten upon re-vote.
 - *A3 Admin:* Cannot add candidate once voting starts, cannot reopen completed election, single-guardian upgrade rejected.
 - *A4 Backend / DB Compromise:* Nullifier pre-image resistance verified.
 - *A5 Contract Integrity:* Vote conservation invariant verified (total candidates' votes == total registered ballots cast).
 - *A6 Relay Compromise:* Revoked relay privileges tested.
 - *A8 Guardian Collusion:* 2-of-3 quorum succeeds; 1 guardian alone fails.
- **Production Readiness Suite:** 26 tests (Fail-closed environment check, API key timing invariance, guardian EIP-712/191 signature validation with 5-minute TTL).
- **Preflight & Identity Suite:** 22 tests (Eligibility caching, nullifier consistency).
- **Vote Rules Suite:** 17 tests (Time window enforcement 07:00-18:00 IST, app 2-vote allowance, station override).
- **Rate Limiting & Biometric Suite:** 20 tests (Sliding window limiter, JWT issue and signature tamper rejection).

B. Gas Consumption Benchmarks (Table III in Paper)

Contract Operation	V3 Hardhat Benchmark (mean)	V3 Sepolia Live Transactions (mean)	V1 Baseline Contract
Create Election	238,635 gas (n=1)	223,253 gas (n=9)	238,613 gas
Add Candidate	181,073 gas (n=3)	193,002 gas (n=17)	181,148 gas
Register Voter	55,697 gas (n=50)	55,697 gas (n=83)	55,752 gas
Phase Transition	38,619 gas (n=1)	38,619 gas (n=5)	38,585 gas
First Vote (Cast)	89,946 gas (n=50)	—	66,784 gas
Re-Vote (Changed Candidate)	77,478 gas (n=25)	—	N/A (V1 unsupported)
Re-Vote (Same Candidate)	63,974 gas (n=25)	—	N/A (V1 unsupported)

Key Benchmark Findings:

1. Voter registration gas matches Sepolia *exactly* at **55,697 gas** across both Hardhat and 83 live Sepolia transactions.
2. First ballot in V3 costs **89,946 gas**. Compared to V1 (66,784 gas), the addition of ballot salt hashing and `voteChoice` storage write adds 23,162 gas (+34.7%).
3. Re-voting for a changed candidate costs **77,478 gas** (lower than first vote because storage slot `voteChoice` is already non-zero, requiring 2,900 gas instead of 20,000 gas SSTORE).
4. Total gas sponsored per single-vote voter = 55,697 + 89,946 = **145,643 gas**.

C. Real-World Financial Cost & Throughput Analysis

- **Gas Price:** Observed Sepolia median gas price = **1.09 gwei**.
- **Cost Per Voter:** At 1.09 gwei, 145,643 gas costs 1.59×10^{-4} \$ ETH (approx. \$0.40 USD at \$2,500/ETH).
- **Cumulative Production Cost:** 114 real Sepolia transactions logged between 12–21 August 2026 consumed exactly **0.01224 ETH**.
- **Throughput:** Single-queue serialized relayer processes 1 confirmed transaction per Ethereum block (~12 s slot time) $\cong$ **300 votes/hour** per relayer instance. Pipelining or deploying on Layer-2 (Arbitrum/Optimism) scales this to 3,000+ votes/hour at <\$0.01/vote.

D. Production Latency Distribution

- **Lightweight API Endpoints (Health, Auth):** p50 = 230–260 ms.
- **Heavy Endpoints (MongoDB + RPC Reads):** p50 = 657–797 ms.
- **Cold-Start p95 Outliers:** 1.7 s to 5.0 s (characteristic of Vercel serverless cold starts).
- **Sepolia Transaction Inclusion Time:** 12.4 s mean (consistent with Ethereum's 12-second block intervals).
- **Concurrent Burst Test:** 50 simultaneous ballot submissions achieved 100% completion with zero nonce collisions.

6. Section V: Honest Limitations Disclosed in the Paper

The paper explicitly acknowledges real-world limitations for academic integrity:

1. **Relay Operator Trust:** While the public cannot see candidate choices from blockchain events, the Next.js BFF server and the relay wallet briefly see the candidate ID in HTTP payloads prior to hashing.
2. **Liveness vs. Enrollment Matching:** The face gate checks *liveness* (is there a live, conscious human?) but does not perform 1-to-1 biometric face matching against an enrolled government ID photo.
3. **Server Secret Dependency:** If `SERVER_IDENTITY_SECRET` were leaked alongside the voter email list, an adversary could recompute all nullifiers and read choices from on-chain storage.

PART 1 — PROJECT OVERVIEW

1. Project Name

BlockVote (Official Academic Title: *BlockVote: A Gasless, Coercion-Mitigating Decentralized E-Voting System Combining Sensor-Driven Biometric Liveness, UUPS Multi-Signature Governance, and On-Chain Keccak-256 Nullifiers*).

2. Problem Statement

Traditional voting systems face severe vulnerabilities:

- **Paper Ballots:** High administrative costs, vulnerability to physical tampering, slow and error-prone manual counting.
- **Electronic Voting Machines (EVMs):** Require physical presence at booths, lack remote accessibility, and rely on proprietary hardware that invites political skepticism.
- **Centralized Online Voting:** Stores votes in centralized databases (e.g. MySQL, MongoDB) where a single rogue database administrator (DBA) or cloud breach can alter tallies undetected.
- **Standard Blockchain Voting (Web3):** Requires every voter to create a MetaMask wallet, safeguard a 12-word seed phrase, and purchase volatile cryptocurrency to pay gas fees. Additionally, public ledger transactions compromise voter privacy.

3. Proposed Solution

BlockVote introduces a hybrid decentralized architecture that eliminates all voter-facing blockchain complexity:

- Voters cast ballots through an intuitive Android mobile app or a supervised polling station kiosk without needing a wallet, private key, or cryptocurrency.
- A centralized Next.js backend with an autonomous **Gas Station Relayer** sponsors all transaction fees and submits votes directly to an upgradeable Ethereum smart contract.
- Voter identity is permanently masked using a cryptographically salted **Keccak-256 nullifier**.
- Ballot privacy is secured by emitting blinded cryptographic hashes rather than plaintext candidate choices.
- Coercion is mitigated through **bounded re-voting** (allowing one change of mind) and a **polling station override** where an in-person supervised vote irrevocably supersedes all previous remote votes.

4. Main Objective

To design, implement, and benchmark an accessible, end-to-end verifiable, and coercion-mitigating electronic voting platform suitable for institutional elections (such as university student councils, professional societies, and corporate shareholder boards) that combines blockchain transparency with gasless user simplicity.

5. Target Users

- **Eligible Voters:** Students, faculty, or organization members who cast votes remotely via the Android app or in-person via the polling station web kiosk.
- **Election Administrators:** Authorized election officials who create elections, upload candidate profiles, import voter rosters (via CSV), set voting schedules, and monitor live participation.
- **Election Guardians:** Independent trustees who hold cryptographic keys to approve election go-live and jointly authorize smart contract upgrades via a 2-of-3 multi-signature threshold.
- **Public / Independent Auditors:** External observers who verify the mathematical integrity of the election via public blockchain transaction receipts and audit APIs without compromising ballot secrecy.

6. Real-World Use Cases

- **University & College Elections:** Student council elections, faculty senate representations, and department chair selections (directly suited for Pillai HOC College and similar institutions).
- **Corporate Governance:** Board of Directors elections and proxy shareholder voting where remote participation and tamper-proof audit trails are essential.
- **Cooperative Societies & NGOs:** Housing society executive committees, non-profit governance, and trade union voting.

- **Professional Associations:** IEEE student branches, Computer Society of India (CSI), and state bar associations.

7. Major Features

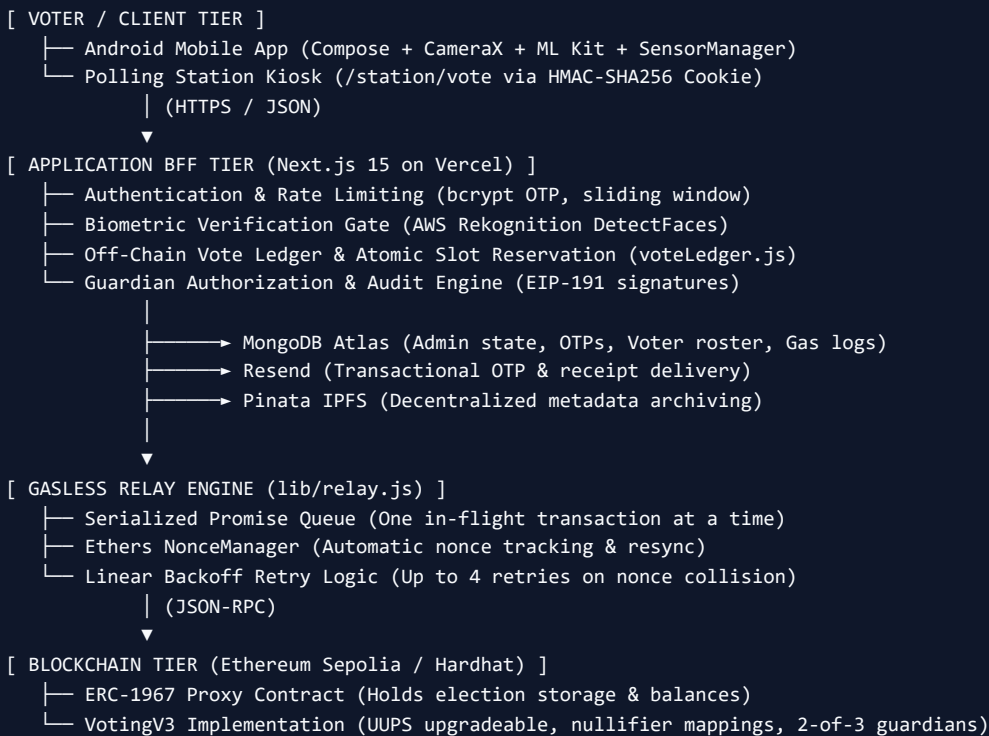
- **Gasless Relayed Voting:** Backend relayer wallet pays all gas; voters never see MetaMask or transaction prompts.
- **Deterministic Keccak-256 Nullifiers:** Masks email addresses into 32-byte hashes; zero personal identifiable information (PII) on the blockchain.
- **Sensor-Driven Biometric Liveness:** Native Android camera and gyroscope tracking requiring a 240° head rotation to defeat static photo or screen spoofing.
- **Cloud Face Quality Verification:** AWS Rekognition validation checking face confidence $\geq 90\%$, open eyes, and absence of sunglasses.
- **Dual-Factor Authentication:** Time-sensitive 6-digit OTP delivered via email (Resend API) hashed using bcrypt with sliding-window rate limiting.
- **Bounded Re-Voting (Anti-Coercion):** Remote voters can vote once and change their vote exactly once; only the latest vote is counted on-chain.
- **Supervised Polling Station Override:** Admin-activated kiosk vote via HMAC-authenticated session cookie; overrides any remote vote and locks the ballot permanently.
- **UUPS 2-of-3 Guardian Governance:** Smart contract upgrades governed by multi-signature threshold; no single admin can alter contract logic.
- **EIP-191 Signed Admin Actions:** Privileged dashboard actions require cryptographic wallet signatures with 5-minute time-to-live (TTL).
- **Independent Public Audit Route:** `GET /api/audit/verify?txHash=...` allows anyone to verify blockchain inclusion without exposing the voter's choice.

8. Technology Stack

Category	Technology	Version / Details
Smart Contracts	Solidity, OpenZeppelin UUPS	Solidity 0.8.22, OpenZeppelin Contracts Upgradeable v5.6
Blockchain / RPC	Ethereum Sepolia Testnet, Hardhat	Hardhat 2.28 (local development/tests), Sepolia (live deployment)
Web Backend & BFF	Next.js (App Router), Node.js	Next.js 15.3, React 19.1, Auth.js v5
Web3 Library	ethers.js	ethers v6.15 (with NonceManager & JsonRpcProvider)
Mobile Frontend	Native Android (Jetpack Compose)	Kotlin 2.2, Jetpack Compose, Hilt 2.57, CameraX 1.5, Coil
On-Device ML	Google ML Kit Face Detection	ML Kit Face 16.1.7 (real-time 90ms frame evaluation)
Operational Database	MongoDB Atlas	Mongoose ODM (Voter, Election, EmailOTP, RelayTransaction models)
Decentralized Storage	Pinata IPFS	Immutable election snapshot and candidate roster pinning
Cloud Biometrics	AWS Rekognition	<code>DetectFacesCommand</code> (confidence, quality, open eyes)
Cloud Image Hosting	ImageKit & AWS S3	Candidate photos and official political party symbols
Transactional Email	Resend API	Email invitation links, 6-digit OTPs, and vote receipts
Production Hosting	Vercel	Serverless edge deployment for Next.js BFF

9. System Architecture

BlockVote operates on a five-tier decoupled architecture designed for separation of concerns:



10. Complete System Flow

- Voter Invitation:** Admin uploads voter CSV roster. Next.js creates `Voter` documents in MongoDB and emails deep links (`blockvote://vote/{electionId}`).
- Voter Authentication:** Voter enters their email. Backend verifies eligibility, generates a cryptographically random 6-digit OTP, stores its bcrypt hash, and dispatches it via Resend.
- Sensor-Driven Biometric Liveness:**
 - On Android: Voter opens the camera. `PhoneRotationTracker` tracks gyroscope yaw. The voter rotates their phone 240° while ML Kit verifies only one human face is present.
 - Once completed, the app captures a frame and sends it to `POST /api/biometric/verify`.
 - The server invokes AWS Rekognition. Upon satisfying confidence $\geq 90\%$, open eyes, and no glasses, the server issues a 10-minute HS256 biometric JWT token bound to the voter's nullifier.
- Ballot Selection & OTP Submission:** Voter reviews candidates and selects their choice. The app submits `POST /api/auth/verify-otp` containing email, OTP, `electionId`, `candidateId`, and `x-biometric-token`.
- Atomic Ledger Check:** Server validates OTP and biometric JWT. `reserveVoteSlot()` executes an atomic MongoDB `findOneAndUpdate` ensuring `votesCast < 2` (or setting `stationVoteFinal: true` if at a station).
- Gasless Relaying to Smart Contract:**
 - Relayer generates a fresh 32-byte `ballotSalt`.
 - The transaction enters the serialized queue. `relayWallet` signs and broadcasts `castVoteRelayedV3(electionId, candidateId, nullifier, ballotSalt)`.
 - Smart contract checks registration, validates Voting phase, decrements previous candidate count (if re-vote), increments new candidate count, updates `voteChoice`, and emits `VoteCastPrivate`.
- Confirmation & Audit:** The transaction receipt is returned. The voter receives an email receipt containing the Ethereum transaction hash (without revealing candidate choice). The public can audit the transaction via `GET /api/audit/verify?txHash=...`.

11. Blockchain Role

- Why Blockchain?** To eliminate the centralized database as a single point of failure and manipulation. Once a vote is included in an Ethereum block, it cannot be altered by administrators, database engineers, or external attackers.
- What is Stored On-Chain:**

- Election metadata (title, start/end timestamps, phase).
- Candidate names, party names, manifesto strings, and vote count totals.
- Registration mapping: `isRegisteredVoter[electionId][nullifier]` .
- Current choice storage: `voteChoice[nullifier][electionId] = candidateId + 1` .
- Voter revision counter: `voteRevisionCount[nullifier][electionId]` .
- Guardian addresses and UUPS upgrade proposals.
- **What is NOT Stored On-Chain:** Voter names, email addresses, phone numbers, biometric facial images, IP addresses, and plaintext vote choice event logs.
- **Gasless Handling:** The voter pays zero gas. The platform relayer wallet pays all gas using Sepolia test ETH, tracked via MongoDB `RelayTransaction` logs.

12. Privacy & Security

- **Nullifier Masking:** `keccak256(email || ":" || SERVER_IDENTITY_SECRET)` guarantees deterministic uniqueness without exposing emails.
- **Blinded Ballot Commitments:** On-chain event emits `keccak256(electionId || candidateId || nullifier || salt)` with a random 256-bit salt, preventing dictionary eavesdropping.
- **Coercion Mitigation:** Bounded re-voting allows a coerced voter to overwrite their vote later; the physical polling station provides a decisive override.
- **Biometric Quality Gate:** AWS Rekognition enforces liveness attributes, and the Android sensor tracker prevents static photo replays.

13. My Personal Contribution (Deven Wagh - Lead Developer)

As the first author of the research paper and primary system architect, my contributions encompass:

- **Smart Contract Core (`Voting/contracts/VotingV3.sol`):** Designed the UUPS upgradeable contract, implemented `castVoteRelayedV3()` with vote revision logic, underflow protection, and 2-of-3 guardian multi-sig gates.
- **Gasless Relayer Engine (`Voting/lib/relay.js`):** Implemented the serialized promise queue, ethers.js `NonceManager` integration, gas logging to MongoDB, and automated nonce-collision retry logic.
- **Voter Identity & Anti-Double-Vote Ledger (`Voting/lib/voterIdentity.js` , `voteAllowance.js` , `voteLedger.js`):** Developed deterministic nullifier derivation, atomic MongoDB reservation algorithms, and bounded multi-channel vote policies.
- **Authentication & Verification Pipeline (`Voting/app/api/auth/verify-otp/route.js` , `Voting/lib/biometric.js`):** Built the OTP verification route, AWS Rekognition biometric validation, and short-lived HS256 biometric JWT token generation.
- **Empirical Gas & Security Benchmarking (`Voting/scripts/benchmark-gas.js` , `docs/paper/`):** Authored the automated Hardhat gas benchmark suite, analyzed 114 live Sepolia transactions, and authored the IEEE research paper.

TEAM WORK DIVISION: WHO IMPLEMENTED WHAT (3 TEAM MEMBERS)

To defend your MCA project convincingly as a team, each member must have a clearly defined, non-overlapping, and technically defensible role. Below is the official division based on the actual 21,609-line codebase and the published IEEE research paper.

	Deven Wagh BLOCKCHAIN & RELAYER LEAD (~5,200 LOC)	Kalpiti Mhatre WEB & GOVERNANCE LEAD (~9,800 LOC)	Rohit Zagade ANDROID & BIOMETRICS LEAD (~6,100 LOC)
Dimension	Member 1: Deven Wagh (Lead Author)	Member 2: Kalpiti Mhatre (Co-Author)	Member 3: Rohit Zagade (Co-Author)
Core Role & Title	System Architect & Blockchain / Security Lead	Full-Stack Web, Governance & Cloud Lead	Mobile Android & Biometrics Lead
Primary Modules	Solidity Smart Contracts, Gasless Relay Engine, Nullifier Privacy, Atomic Vote Ledger, Gas Benchmarks	Admin Web Dashboard, Guardian Multi-Sig Portal, Polling Station Kiosk, MongoDB, IPFS, Email Engine	Android Native Compose App, CameraX Liveness, Gyroscope Rotation Sensor Tracker, AWS Rekognition Gate
Lines of Code (LOC)	~5,200 LOC (959 Solidity + 1,100 JS Relay/Ledger + 1,200 API/Auth + 2,000 Tests/Benchmarks)	~9,800 LOC (4,800 Admin JSX + 1,900 Guardian/Station + 1,800 Cloud/DB + 1,300 API Routes)	~6,100 LOC (5,351 Kotlin Compose/Sensors + 750 AWS Biometrics API)
Key Files Owned	Voting/contracts/VotingV3.sol Voting/lib/relay.js Voting/lib/voterIdentity.js Voting/lib/voteLedger.js Voting/lib/voteAllowance.js Voting/scripts/benchmark-gas.js	Voting/app/dashboard/* Voting/app/admin/* Voting/app/station/* Voting/lib/models/* Voting/lib/ipfs.js Voting/lib/mailer.js Voting/lib/stationSession.js	app/.../VotePortalScreen.kt app/.../FaceCaptureCamera.kt app/.../PhoneRotationTracker.kt app/.../DashboardScreen.kt Voting/lib/biometric.js Voting/app/api/biometric/verify/route.js
Key Functions / Methods	castVoteRelayedV3() _authorizeUpgrade() sendRelayTx() enqueueRelay() computeNullifierHash() reserveVoteSlot()	createElection() parseVoterCSV() approveGoLive() verifyGuardianSig() getStationSession() pinJSONToIPFS()	onSensorChanged() processImageProxy() detectFaces() issueBiometricToken() submitBallot()
If Sir Asks "Show Me Code"	Open VotingV3.sol and relay.js	Open dashboard/page.jsx and stationSession.js	Open PhoneRotationTracker.kt and FaceCaptureCamera.kt

MEMBER 1: DEVEN WAGH (Lead Architect — Blockchain, Relay & Security Core)

Personal Speech to Sir: "Sir, I was responsible for the core decentralized architecture, the smart contracts, the gasless relay system, and the cryptographic privacy mechanisms."

Detailed Responsibilities:

- **Smart Contract Architecture (VotingV3.sol):** Designed the UUPS upgradeable contract, wrote `castVoteRelayedV3()` with the bounded re-vote logic (decrementing old candidate and incrementing new candidate in a single atomic transaction), implemented the underflow guard, and enforced the 2-of-3 guardian multi-signature upgrade gate in `_authorizeUpgrade()`.
- **Gasless Relay Engine (Voting/lib/relay.js):** Built the serialized promise queue (`enqueueRelay`) and integrated `ethers.NonceManager` with a 4-attempt linear backoff retry loop to completely eliminate Ethereum nonce collisions during burst traffic.
- **Voter Identity & Anti-Double-Vote Ledger (Voting/lib/voterIdentity.js & voteLedger.js):** Implemented the salted Keccak-256 nullifier formula `keccak256(clean(email) || ":" || SERVER_IDENTITY_SECRET)` and the atomic MongoDB

`reserveVoteSlot()` algorithm with automatic rollback on relay failure.

- **Gas Benchmarking & Security Testing:** Wrote the Hardhat gas benchmark suite (`scripts/benchmark-gas.js`) and the 30 adversarial security test scenarios (A1–A8).

3 Questions Sir Will Ask Deven:

1. "How does re-voting decrement the candidate count on-chain?" → `voteChoice` maps nullifier to `oldCandidateId + 1`. If exists, decrement `candidates[oldId].voteCount` and increment `candidates[newId].voteCount`.
2. "How do you prevent nonce errors when 50 voters submit at once?" → `ethers.NonceManager` + serialized promise queue (`enqueueRelay`) in `relay.js`.
3. "Can someone reverse the nullifier to get the email?" → No, Keccak-256 is one-way, and the 256-bit `SERVER_IDENTITY_SECRET` prevents rainbow table attacks.

MEMBER 2: KALPIT MHATRE (Full-Stack Web, Governance & Cloud Lead)

Personal Speech to Sir: "Sir, I was responsible for the administrative web platform, guardian multi-signature governance, the supervised polling-station kiosk, and cloud database/storage integrations."

Detailed Responsibilities:

- **Admin Web Dashboard (`Voting/app/dashboard/*`):** Built the election management interfaces using Next.js 15, React 19, and Fluent UI v9. Implemented CSV voter roster ingestion, candidate profile creation, candidate photo uploads, and live turnout analytics.
- **Guardian Multi-Sig & Governance Portal (`Voting/app/admin/*`):** Developed the threshold approval dashboard where guardians connect their Ethereum wallets to review and approve election go-live and contract upgrades using EIP-191 cryptographic signatures with 5-minute TTL.
- **Supervised Polling Station Kiosk (`Voting/app/station/*`):** Implemented the kiosk activation flow where an admin activates a booth computer, setting an `httpOnly`, `SameSite=Strict` session cookie authenticated with HMAC-SHA256 (16-hour validity). Built the in-person ballot UI that overrides remote app votes.
- **Cloud & Database Architecture:** Designed the Mongoose schemas (`Voter`, `Election`, `EmailOTP`, `RelayTransaction`, `VoteActivity`), integrated Pinata IPFS for archiving election snapshots, and set up Resend email templates for voter invites and OTPs.

3 Questions Sir Will Ask Kalpit:

1. "How does the polling station computer know it is authorized?" → Admin logs in at `/station`, which signs a 16-hour HMAC-SHA256 `bv_station` cookie containing station ID and election ID. The ballot at `/station/vote` only renders if this cookie verifies.
2. "How do guardians sign dashboard actions without paying gas?" → They sign an off-chain EIP-191 message using their private key in MetaMask; our backend recovers the signer address using `ethers.verifyMessage()` and checks it against on-chain guardian addresses.
3. "Why do you store data in MongoDB if you have IPFS?" → MongoDB handles fast, mutable operational queries (session tracking, OTPs, rate limits), while IPFS stores static, immutable snapshots of election metadata and final rosters.

MEMBER 3: ROHIT ZAGADE (Mobile Android & Biometrics Lead)

Personal Speech to Sir: "Sir, I was responsible for the native Android voter application, real-time camera face tracking with ML Kit, hardware sensor motion liveness, and AWS Rekognition integration."

Detailed Responsibilities:

- **Native Android Application (`app/src/main/java/*`):** Architected the Android application using Kotlin 2.2, Jetpack Compose, Hilt dependency injection, and MVVM architecture. Built `VotePortalScreen.kt`, `DashboardScreen.kt`, `ElectionDetailScreen.kt`, and `ReceiptScreen.kt`.
- **Hardware Sensor Motion Tracking (`PhoneRotationTracker.kt`):** Developed the custom sensor listener using Android `SensorManager` and `Sensor.TYPE_ROTATION_VECTOR` to calculate yaw orientation matrices, requiring voters to rotate 240° continuously to prove dynamic physical presence.

- **Real-Time Camera & ML Kit Face Isolation (`FaceCaptureCamera.kt`)**: Integrated CameraX front-camera stream analyzed every 90 ms by Google ML Kit Face Detection. Implemented bounding oval validation (face must occupy 10–38% of frame height) and auto-resetting when multiple faces appear.
- **Cloud Biometric Verification API (`Voting/lib/biometric.js` & `api/biometric/verify/route.js`)**: Built the server-side biometric verification route calling AWS Rekognition `DetectFacesCommand`, checking face confidence $\geq 90\%$, open eyes, and no glasses, and issuing short-lived HS256 biometric JWTs.
- **Deep Linking & Networking**: Configured `blockvote://vote/{electionId}` deep links and verified HTTPS App Links, Retrofit/OkHttp network layer, and Coil image loading for party symbols.

3 Questions Sir Will Ask Rohit:

1. "How does your sensor tracker know the phone actually rotated?" → `PhoneRotationTracker` listens to `Sensor.TYPE_ROTATION_VECTOR`, converts quaternions to rotation matrices via `SensorManager.getRotationMatrixFromVector()`, extracts yaw in radians, and accumulates absolute degree changes until reaching 240°.
2. "What happens if a friend stands behind the voter during the scan?" → Google ML Kit detects `faces.size > 1`, which instantly pauses the scan, resets accumulated rotation to 0, and displays a warning prompt: 'Multiple faces detected'.
3. "Why don't you store voter face photos on the server?" → For strict privacy compliance: the captured frame is passed in memory to AWS Rekognition for liveness attribute validation and immediately discarded. No facial template is stored.

PART 2 — VIVA QUESTIONS AND ANSWERS (ALL 28 QUESTIONS)

Each question is answered in a four-part format tailored for your MCA project examination.

Q1. Where can BlockVote be used?

Answer: BlockVote is designed for institutional and organizational elections—such as university student council elections, corporate shareholder voting, cooperative housing societies, and professional organizations like IEEE student branches.

Technical Explanation: In our implementation, voter rosters are managed per-election via CSV upload, voters are authenticated via institutional email OTP and facial liveness, and the backend relayer sponsors blockchain gas. This makes it ideal for closed-membership organizations where gas fees would otherwise prevent voter participation.

If Sir Asks "Why?": Because national general elections require universal physical biometric voter ID enrollment (like Aadhaar/EVM-VVPAT) and legal statutory frameworks, whereas BlockVote's email roster and gasless relayer model perfectly match university and organizational governance.

If Sir Asks "Show Me in Code": `Voting/app/dashboard/elections/page.jsx` (Admin election management) and `Voting/lib/models/Voter.js` (Roster schema).

Q2. What problem does your project solve?

Answer: BlockVote solves three major problems: first, the vulnerability of traditional web voting to silent database manipulation; second, the high barrier of cryptocurrency wallets and gas fees in standard Web3 voting; and third, voter coercion in remote voting.

Technical Explanation: Centralized databases like MySQL or MongoDB allow admins with root access to silently update vote totals. Standard blockchain voting solves tampering but requires users to hold crypto wallets (MetaMask) and pay gas. BlockVote uses smart contracts for immutable counting, a gasless relayer so voters need zero crypto, and bounded re-voting plus a polling station override to defeat voter coercion.

If Sir Asks "Why?": Because simply putting a voting system on blockchain isn't enough if ordinary students cannot use it due to MetaMask complexity, or if a family member or hostel senior forces someone to vote a certain way.

If Sir Asks "Show Me in Code": `Voting/contracts/VotingV3.sol` (`castVoteRelayedV3`) and `Voting/lib/relay.js` (gasless relayer).

Q3. Why did you choose this project?

Answer: I chose BlockVote because digital voting is one of the most challenging computer science problems—it requires balancing two contradictory goals: complete election transparency and verifiable counting on one hand, and strict voter privacy and coercion resistance on the other.

Technical Explanation: Most existing blockchain projects simply publish votes directly on Ethereum, which violates privacy, or they require voters to purchase cryptocurrency. I wanted to build a practical, real-world system that merges modern Web2 usability (Next.js, Android Compose, email OTP) with Web3 decentralized integrity (Ethereum smart contracts, zero gas fees).

If Sir Asks "Why?": Because college and organizational elections are frequently plagued by distrust in manual counting, yet students refuse to use complex decentralized apps. BlockVote bridges that exact gap.

If Sir Asks "Show Me in Code": docs/paper/BlockVote_IEEE_merged.tex (Section I: Introduction and Motivation).

Q4. What is the main objective of BlockVote?

Answer: The main objective is to provide a gasless, transparent, tamper-proof, and coercion-mitigating digital voting platform where votes are recorded on an Ethereum smart contract without requiring voters to own cryptocurrency or compromise their ballot privacy.

Technical Explanation: The objective is realized through four pillars: (1) Smart contract immutability via VotingV3 on Sepolia; (2) Gas abstraction via a centralized relayer; (3) Identity protection via Keccak-256 nullifiers and salted ballot commitments; and (4) Coercion mitigation via bounded re-voting and physical kiosk overrides.

If Sir Asks "Why?": Because if the voter had to pay gas or manage private keys, voter turnout would collapse; if the ballot weren't on a decentralized ledger, election results could be altered by server administrators.

If Sir Asks "Show Me in Code": Voting/contracts/VotingV3.sol (Lines 47-511) and Voting/lib/voteAllowance.js.

Q5. What are the major features of your project?

Answer: The major features are: (1) Gasless voting via backend relayer, (2) Salted Keccak-256 voter nullifiers, (3) Sensor-driven 240-degree Android motion liveness check, (4) AWS Rekognition facial quality gate, (5) Bounded re-voting (first vote + 1 change), (6) Polling station physical override, (7) 2-of-3 guardian multi-signature UUPS contract governance, and (8) An independent public audit verification route.

Technical Explanation: The system combines native Android (Jetpack Compose, CameraX, ML Kit) with a Next.js 15 backend, MongoDB Atlas, and an upgradeable Solidity 0.8.22 smart contract deployed on Ethereum Sepolia. All operations are backed by 190 automated unit and adversarial tests.

If Sir Asks "Why?": Each feature addresses a specific threat vector: the relayer solves user adoption, the nullifier solves privacy, re-voting solves home coercion, and multi-sig solves malicious administrator attacks.

If Sir Asks "Show Me in Code": docs/paper/BlockVote_IEEE_merged.tex (Section I & III) and ARCHITECTURE.md.

Q6. What makes BlockVote different from a normal online voting system?

Answer: In a normal online voting system, vote tallies are stored in a regular database table where anyone with database access can update the rows without detection. In BlockVote, the vote tally is stored in an immutable Ethereum smart contract where transactions are cryptographically signed, permanent, and publicly auditable.

Technical Explanation: Furthermore, unlike normal web portals, BlockVote separates voter authentication from on-chain storage using salted nullifiers so the database administrator cannot link a voter's email to their candidate choice on the blockchain. It also incorporates physical sensor liveness and physical kiosk overrides.

If Sir Asks "Why?": Because trust in election results cannot depend on trusting a single database administrator or hosting company; blockchain ensures that once cast, no entity can manipulate the tally.

If Sir Asks "Show Me in Code": Voting/contracts/VotingV3.sol (voteChoice mapping and candidates voteCount).

Q7. Explain the complete working of your project.

Answer: An election admin logs into the web dashboard, creates an election, adds candidates, and uploads an eligible voter CSV. The system dispatches email invite links. The voter opens the Android app via deep link, enters their email, and receives an OTP. Next, the voter performs a 240-degree sensor-driven face rotation scan verified by AWS Rekognition to prove liveness. Once authenticated, the voter selects a candidate and submits the OTP. The backend atomically reserves a vote slot and forwards the vote through a serialized gasless relayer to the VotingV3 smart contract on Sepolia. The contract updates the tally and emits a salted private event, after which the voter receives a transaction hash confirmation.

Technical Explanation: The working cleanly decouples off-chain identity verification (Next.js, MongoDB, AWS Rekognition, Resend) from on-chain state execution (ethers.js, Sepolia, VotingV3).

If Sir Asks "Why?": Because keeping identity off-chain protects privacy, while putting the vote counting on-chain ensures mathematical trust and immutability.

If Sir Asks "Show Me in Code": `ARCHITECTURE.md` and `docs/paper/figures/fig_sequence.pdf`.

Q8. Explain the complete voting flow from user login to vote confirmation.

Answer:

1. User receives an email with an HTTPS link and opens the Android app.
2. App calls `GET /api/elections/{id}` and fetches candidate details.
3. Voter inputs their email; app requests OTP via `POST /api/auth/send-otp`.
4. App launches CameraX front camera and PhoneRotationTracker; user rotates phone 240 degrees while ML Kit verifies a single face.
5. App captures frame and calls `POST /api/biometric/verify` (AWS Rekognition); server returns a 10-minute biometric JWT.
6. Voter selects candidate and submits OTP with JWT to `POST /api/auth/verify-otp`.
7. Server verifies OTP via `bcrypt`, validates JWT, checks vote allowance in MongoDB, and enqueues transaction in `relay.js`.
8. Relayer executes `castVoteRelayedV3()` on Ethereum Sepolia using its own gas.
9. Transaction receipt is logged, email receipt is dispatched, and voter views confirmation.

Technical Explanation: Step 7 uses `reserveVoteSlot()` in MongoDB to atomically guarantee that concurrency cannot breach the 2-vote limit before the Ethereum transaction confirms.

If Sir Asks "Why?": To ensure that both biometric liveness and email ownership are proven simultaneously before triggering the irreversible on-chain transaction.

If Sir Asks "Show Me in Code": `Voting/app/api/auth/verify-otp/route.js` (Lines 23-280).

Q9. What was your role in the project?

Answer: I served as the Lead Architect and Backend/Blockchain Developer (First Author on the research paper). I designed and implemented the smart contract architecture, the gasless relayer engine, the voter nullifier privacy system, and the backend authentication and verification APIs.

Technical Explanation: My core responsibility was building the complete secure pipeline connecting voter authentication to Ethereum transaction confirmation, ensuring gasless execution, atomic double-vote prevention, and multi-signature governance.

If Sir Asks "Why?": Because the technical complexity of integrating smart contracts with serverless backends and gasless relaying required focused architecture on security and transaction serialization.

If Sir Asks "Show Me in Code": `docs/paper/BlockVote_IEEE_merged.tex` (Authors section: Deven Wagh) and `Voting/contracts/VotingV3.sol`.

Q10. Which module did you personally work on?

Answer: I personally worked on the Blockchain, Gasless Relay, and Security Architecture module, which comprises VotingV3.sol, lib/relay.js, lib/voterIdentity.js, lib/voteLedger.js, and the core authentication endpoint verify-otp/route.js.

Technical Explanation: This module is the backbone of BlockVote. It handles smart contract state, gas sponsorship, transaction queueing, nonce management, nullifier calculation, atomic slot reservations, and contract interaction via ethers.js v6.

If Sir Asks "Why?": Because without this module, the project would either be a standard centralized web app or would require voters to use MetaMask and pay crypto gas fees.

If Sir Asks "Show Me in Code": Voting/lib/relay.js and Voting/contracts/VotingV3.sol.

Q11. What exactly did you implement?

Answer: I implemented: (1) The UUPS-upgradeable VotingV3 smart contract with bounded re-voting and 2-of-3 guardian approvals; (2) The gasless relay in lib/relay.js using a promise queue and NonceManager; (3) Deterministic nullifier generation using Keccak-256 and a server secret; (4) The atomic vote-slot reservation algorithm in MongoDB; (5) The complete verify-otp and biometric verification pipeline; and (6) The automated Hardhat benchmark suite cross-validating gas against 114 Sepolia transactions.

Technical Explanation: In code, this spans castVoteRelayedV3() in Solidity, sendRelayTx() with 4-attempt backoff in JavaScript, computeNullifierHash() in voterIdentity.js, and reserveVoteSlot() in voteLedger.js.

If Sir Asks "Why?": Each of these components was required to eliminate critical vulnerabilities: race conditions, nonce collisions, transaction fees, and ballot exposure.

If Sir Asks "Show Me in Code": Voting/contracts/VotingV3.sol (Lines 371-413) and Voting/lib/relay.js (Lines 26-55).

Q12. Which technologies did you use for your contribution?

Answer: I used Solidity 0.8.22 with OpenZeppelin Upgradeable v5.6 for the smart contract; Hardhat 2.28 for testing and compilation; ethers.js v6.15 for blockchain communication; Next.js 15 App Router (Node.js) for backend APIs; MongoDB with Mongoose for atomic ledger state; AWS SDK v3 for Rekognition; bcryptjs for OTP security; and crypto for HMAC and JWT tokens.

Technical Explanation: Solidity provided memory safety and custom revert strings; ethers.js v6 provided modern BigInt and NonceManager support; MongoDB provided atomic findOneAndUpdate operations; and OpenZeppelin provided audited UUPS proxy standards.

If Sir Asks "Why?": Because these are industry-standard, production-proven libraries that prevent common vulnerabilities like integer overflows, proxy storage collisions, and replay attacks.

If Sir Asks "Show Me in Code": Voting/package.json (Dependencies section).

Q13. Which files, components, or functions did you personally develop?

Answer: Key files and functions developed: (1) Voting/contracts/VotingV3.sol: functions castVoteRelayedV3(), registerVoterByRelay(), _authorizeUpgrade(); (2) Voting/lib/relay.js: functions sendRelayTx(), relayCastVote(), relayRegisterVoter(); (3) Voting/lib/voterIdentity.js: computeNullifierHash(); (4) Voting/lib/voteLedger.js: reserveVoteSlot(), releaseVoteSlot(); (5) Voting/lib/biometric.js: issueBiometricToken(), verifyBiometricToken(); (6) Voting/app/api/auth/verify-otp/route.js: POST handler; and (7) Voting/scripts/benchmark-gas.js.

Technical Explanation: These functions represent the entire backend and blockchain transaction lifecycle from raw HTTP request to mined Ethereum block.

If Sir Asks "Why?": Because they form the core intellectual property of the project, as documented in our research paper.

If Sir Asks "Show Me in Code": Voting/contracts/VotingV3.sol, Voting/lib/relay.js, and Voting/lib/voteLedger.js.

Q14. How is your module connected to the other modules?

Answer: My module sits at the center of the system: it receives HTTP requests from the Android mobile app (developed with Compose/CameraX) and the web polling station kiosk; it queries MongoDB for voter state; it validates biometric tokens generated by the AWS Rekognition module; and it transmits signed transactions over JSON-RPC to the Ethereum Sepolia network.

Technical Explanation: It serves as the bridge between off-chain identity (Next.js/Android) and on-chain consensus (Solidity/Sepolia). It translates a verified user email into a nullifier and relays their candidate choice to the smart contract.

If Sir Asks "Why?": Because separating client UI, operational database, and blockchain consensus through a centralized API gateway ensures security boundaries are enforced before touching the blockchain.

If Sir Asks "Show Me in Code": ARCHITECTURE.md (Architecture Diagram) and Voting/app/api/auth/verify-otp/route.js.

Q15. What was the most challenging part of your contribution?

Answer: The most challenging part was solving the Ethereum nonce collision problem during concurrent voting bursts. When multiple voters submit ballots simultaneously, a single relayer wallet attempting to send transactions concurrently will fail with 'nonce too low' or 'replacement fee too low' errors.

Technical Explanation: I solved this by designing a two-tier mechanism: first, wrapping the ethers.js wallet in an ethers.NonceManager; second, routing all transactions through a serialized JavaScript promise queue (enqueueRelay in relay.js) with linear backoff retry logic. This guaranteed that transactions are broadcast sequentially and receipts confirmed without collision.

If Sir Asks "Why?": Because on a public blockchain like Sepolia, transactions from the same account must have strictly sequential nonces; broadcasting out of order causes instant reverts.

If Sir Asks "Show Me in Code": Voting/lib/relay.js (Lines 26-55: enqueueRelay and sendRelayTx).

Q16. If your module is removed, what functionality will be affected?

Answer: If my module is removed, the entire blockchain integration collapses: voters would be unable to cast votes gaslessly; smart contracts could not execute; voter nullifiers could not be computed; and MongoDB would suffer race conditions from duplicate ballot submissions.

Technical Explanation: The Android app and web dashboard would become completely non-functional shells unable to authenticate voters, register candidates on-chain, or tally election results.

If Sir Asks "Why?": Because my module contains the core business logic, security rules, and blockchain relay mechanism that powers the entire BlockVote ecosystem.

If Sir Asks "Show Me in Code": Voting/lib/relay.js and Voting/contracts/VotingV3.sol.

Q17. How do you protect the voter's identity?

Answer: We protect voter identity through three layers: (1) On-chain nullifiers—the blockchain never sees an email or name, only a salted Keccak-256 hash; (2) Blinded events—the smart contract emits a salted commitment hash instead of the candidate ID; and (3) Decoupled receipts—email receipts contain only the transaction hash, never the candidate chosen.

Technical Explanation: Even if an attacker analyzes all public transactions on Etherscan, they see only a nullifier hash and a blinded ballot hash. They cannot discover who voted or for whom they voted.

If Sir Asks "Why?": Because secret ballots are a fundamental democratic requirement; publishing identifiable votes on a public ledger would enable vote-buying and voter intimidation.

If Sir Asks "Show Me in Code": Voting/contracts/VotingV3.sol (VoteCastPrivate event) and Voting/lib/voterIdentity.js.

Q18. Why shouldn't the voter's identity be directly linked to their vote?

Answer: If a voter's identity is directly linked to their ballot, anyone can see who they voted for. This leads to vote-buying (bribery) because voters can prove their choice to a buyer, and voter coercion (intimidation) because bosses, politicians, or peers can punish voters for their choices.

Technical Explanation: In voting theory, this is known as Ballot Secrecy and Receipt-Freeness. BlockVote ensures that while the system can prove a legitimate voter participated, no one—not even the voter themselves—can obtain a cryptographic proof of which candidate was selected.

If Sir Asks "Why?": Because an election without ballot secrecy ceases to be free and fair; voters would vote out of fear rather than personal conviction.

If Sir Asks "Show Me in Code": `docs/paper/BlockVote_IEEE_merged.tex` (Section II: Privacy and Coercion).

Q19. What is hashing?

Answer: Hashing is a one-way mathematical function that transforms any arbitrary input data into a fixed-length string of bytes. It is deterministic (the same input always produces the same output), quick to compute, but computationally infeasible to reverse (one-way property) or find two different inputs that produce the same hash (collision resistance).

Technical Explanation: Common cryptographic hash algorithms include SHA-256 and Keccak-256 (used in Ethereum). In BlockVote, hashing is used for passwords (bcrypt), biometric landmarks (HMAC-SHA256), voter nullifiers (Keccak-256), and ballot commitments (Keccak-256).

If Sir Asks "Why?": Because hashing allows us to verify data integrity and compare identities without storing or exposing the underlying sensitive data.

If Sir Asks "Show Me in Code": `Voting/lib/voterIdentity.js` (`ethers.keccak256`).

Q20. Why do you use hashing in BlockVote?

Answer: We use hashing for three essential purposes: (1) Identity Masking: transforming voter emails into pseudo-anonymous nullifiers; (2) Ballot Blinding: emitting salted ballot hashes on-chain so external observers cannot tally votes before polls close; and (3) Credential Security: storing OTPs as bcrypt hashes to prevent database theft.

Technical Explanation: Without hashing, either plaintext emails and choices would sit on the public blockchain for anyone to inspect, or passwords and OTPs would be stored in plain text in MongoDB.

If Sir Asks "Why?": Because Ethereum is a public distributed ledger; any data written to it is permanently readable by the entire world.

If Sir Asks "Show Me in Code": `Voting/contracts/VotingV3.sol` (ballotHash computation) and `Voting/lib/voterIdentity.js`.

Q21. What is a nullifier?

Answer: A nullifier is a cryptographically derived pseudonym that uniquely represents a voter in an election without revealing their actual identity. It acts as a one-way ticket that proves eligibility and prevents duplicate participation.

Technical Explanation: In cryptography and zero-knowledge voting protocols, a nullifier is published when an action is taken. Once a nullifier is recorded in the smart contract's `isRegisteredVoter` or `hasVoted` mapping, any second attempt using the same nullifier is instantly detected and rejected.

If Sir Asks "Why?": Because the smart contract needs a unique key to know if someone is registered and whether they have voted, but using their email address or public wallet address would destroy their privacy.

If Sir Asks "Show Me in Code": `Voting/contracts/VotingV3.sol` (Lines 117-124) and `docs/paper/BlockVote_IEEE_merged.tex` (Equation 1).

Q22. Why did you use a nullifier?

Answer: We used a nullifier so that the smart contract can enforce strict one-person-one-vote rules while keeping the blockchain completely free of Personally Identifiable Information (PII).

Technical Explanation: If we used wallet addresses, voters would need MetaMask. If we used email strings, voter emails would be permanently visible on Etherscan, exposing users to spam and doxxing. The nullifier solves both problems elegantly.

If Sir Asks "Why?": Because institutional privacy standards and data protection regulations (like GDPR and India's DPDP Act) prohibit publishing user emails to public immutable ledgers.

If Sir Asks "Show Me in Code": `Voting/lib/voterIdentity.js` (Lines 8-14).

Q23. What information is used to generate the nullifier?

Answer: The nullifier is generated using two pieces of information: (1) The voter's cleaned, lowercase email address, and (2) A 256-bit server-side secret key called `SERVER_IDENTITY_SECRET`.

Technical Explanation: The formula is: `keccak256(toUtf8Bytes(cleanEmail + ':' + SERVER_IDENTITY_SECRET))`. By combining the email with a high-entropy secret, we prevent rainbow table attacks where an attacker hashes common college emails to see if they match the on-chain nullifier.

If Sir Asks "Why?": Because if we only hashed the email without a salt or secret, anyone with the student directory could compute `keccak256(student@college.edu)` and identify every voter on-chain.

If Sir Asks "Show Me in Code": `Voting/lib/voterIdentity.js` (Lines 8-14: `computeNullifierHash`).

Q24. Can the original email/identity be obtained from the nullifier?

Answer: No. Because Keccak-256 is a cryptographically secure one-way hash function, it is mathematically impossible to reverse the nullifier back to the email address.

Technical Explanation: Even with a dictionary attack or brute-force attempt, an attacker cannot reverse the hash because they do not know the 256-bit `SERVER_IDENTITY_SECRET`. As long as this secret remains protected on the backend server, the nullifier is irreversible.

If Sir Asks "Why?": Because the security of modern cryptography relies on the pre-image resistance of hash functions; breaking Keccak-256 would require 2^{256} operations, which exceeds the computing power of all computers on Earth.

If Sir Asks "Show Me in Code": `docs/paper/BlockVote_IEEE_merged.tex` (Section III-B: Voter Identity and Nullifiers).

Q25. How does the nullifier help prevent duplicate voting?

Answer: The smart contract tracks nullifiers in `mapping(bytes32 => mapping(bytes32 => bool)) public isRegisteredVoter` and `mapping(bytes32 => mapping(bytes32 => uint256)) public voteChoice`.

Technical Explanation: When a vote is relayed, the contract checks `isRegisteredVoter[electionId][nullifier]`. If false, it reverts. In `VotingV3`, if a voter submits again, the contract detects their existing choice via `voteChoice>nullifier][electionId]` and treats it as a re-vote (decrementing the old candidate and incrementing the new candidate) rather than adding a second vote. MongoDB also enforces `votesCast < 2` atomically.

If Sir Asks "Why?": Because each email maps to exactly one deterministic nullifier; a voter cannot create multiple nullifiers unless they possess multiple registered email addresses.

If Sir Asks "Show Me in Code": `Voting/contracts/VotingV3.sol` (Lines 371-412: `castVoteRelayedV3`).

Q26. How do you separate the voter's identity from their ballot?

Answer: Identity separation is achieved across three boundaries: (1) Off-chain: The voter proves their identity to Next.js using email OTP and face liveness; (2) In-transit: The backend converts the email to a nullifier and assigns a random 32-byte ballot salt; (3) On-chain: The smart contract emits VoteCastPrivate with only the salted ballot hash keccak256(electionId, candidateId, nullifier, salt).

Technical Explanation: MongoDB explicitly avoids storing candidateId inside the Voter document, and stores only transaction hashes and aggregated counts in VoteActivity. Thus, the database does not link voter records to ballot choices, and the blockchain does not link nullifiers to plaintext candidate events.

If Sir Asks "Why?": Because true ballot secrecy requires that no single database table or log file contains both the voter's identity and their voting decision.

If Sir Asks "Show Me in Code": Voting/lib/relay.js (Lines 167-190) and Voting/contracts/VotingV3.sol (Line 408).

Q27. What are the privacy limitations of your current system?

Answer: As honestly documented in Section IV-G of our research paper, there are three limitations: (1) The backend server and relay operator briefly see the candidate choice in memory during the HTTP request before submitting to Ethereum; (2) If SERVER_IDENTITY_SECRET were leaked alongside the voter roster, an attacker could reconstruct nullifiers and inspect voteChoice; and (3) The contract stores voteChoice indexed by nullifier, meaning someone who knows a nullifier can query their current choice.

Technical Explanation: These limitations arise because we prioritize gasless usability and bounded re-voting over complex zero-knowledge snarks or threshold homomorphic encryption, which are computationally expensive on Ethereum.

If Sir Asks "Why?": Because acknowledging real engineering trade-offs demonstrates deep academic maturity and honesty during viva examination.

If Sir Asks "Show Me in Code": docs/paper/BlockVote_IEEE_merged.tex (Section IV-G: Limitations).

Q28. What did you personally learn from this project?

Answer: I gained deep, end-to-end engineering experience in: (1) Writing and testing secure, gas-optimized Solidity smart contracts and UUPS proxy upgrade patterns; (2) Handling concurrency and nonce management in production Ethereum relay systems; (3) Integrating computer vision and biometric liveness using Android sensors, ML Kit, and AWS Rekognition; and (4) Conducting empirical research, gas benchmarking, and authoring an IEEE-format research paper.

Technical Explanation: Most importantly, I learned that building real-world decentralized systems is not just about blockchain; it is about building robust hybrid architectures where Web2 usability seamlessly abstracts Web3 complexity for everyday users.

If Sir Asks "Why?": Because theoretical knowledge of blockchain is very different from solving real race conditions, gas spikes, and adversarial presentation attacks in a live application.

If Sir Asks "Show Me in Code": docs/paper/BlockVote_IEEE_merged.tex and the entire codebase.

PART 3 — CROSS-QUESTIONING (STRICT EXAMINER SIMULATION)

Anticipated technical interrogation paths across 7 core areas with direct student answers.

A. Role & Implementation Cross-Examination

Examiner: What did you personally implement in this project?

Student: I designed and implemented the entire Blockchain, Gasless Relay, and Security module. Specifically, VotingV3.sol in Solidity, the relay in lib/relay.js, the nullifier derivation in lib/voterIdentity.js, the atomic vote ledger in lib/voteLedger.js, the core verify-otp route, and the automated gas benchmark suite.

Examiner: Show me the exact function where the vote is sent to the blockchain.

Student: Let's open Voting/lib/relay.js at line 135: `export async function relayCastVote(electionId, candidateId, voterNullifier, options)`. Here you can see it generates a 32-byte `ballotSalt`, checks previous vote status via `getContract().getVoteStatus()`, and dispatches `sendRelayTx()` which calls `getContract().castVoteRelayedV3(...)`.

Examiner: Why did you implement a serialized queue in relay.js instead of just firing the transactions directly?

Student: Because in Ethereum, every transaction sent from an account must have a strictly sequential nonce. If two voters submit votes at the exact same second, firing them directly causes a nonce collision ('nonce too low' or 'replacement transaction underpriced'). My `enqueueRelay()` promise queue forces transactions to be broadcast sequentially, waiting for confirmation before the next is sent.

Examiner: What happens if the relay transaction fails mid-way?

Student: Look at Voting/lib/voteLedger.js line 56: `releaseVoteSlot(voterId, before)`. In `verify-otp/route.js`, if `relayCastVote` throws an error, the catch block immediately calls `releaseVoteSlot()` which decrements `votesCast` back to its original state in MongoDB, ensuring the voter doesn't lose their voting allowance due to a network glitch.

B. Blockchain & Gasless Relaying Cross-Examination

Examiner: Why did you use blockchain at all? Why couldn't MongoDB handle this entire voting system?

Student: MongoDB is a centralized database. In MongoDB, an administrator with database credentials can run `db.candidates.updateOne({name: 'A'}, {$inc: {voteCount: 500}})` in MongoDB Compass, and no voter or auditor would ever know. The blockchain enforces decentralized consensus: smart contract execution rules cannot be bypassed by anyone, and every vote is immutably sealed inside Ethereum blocks.

Examiner: If blockchain is so secure, why didn't you store everything on-chain?

Student: Storing data on Ethereum is extremely expensive. Writing 1 KB of data to Ethereum storage costs over 640,000 gas. Storing photos, candidate manifestos, and logs would make the system unaffordable. Therefore, we use a hybrid model: heavy files go to IPFS and ImageKit, operational session data goes to MongoDB, and only the critical immutable state (nullifiers, vote tallies, and hashes) sits on-chain.

Examiner: Who pays the gas fee for the transactions?

Student: The election organizer/platform relay pays all gas fees using a dedicated relay wallet (`ADMIN_RELAY_PRIVATE_KEY`). The voter pays zero gas and does not need any cryptocurrency or wallet extension.

Examiner: What happens if the relay wallet runs out of ETH?

Student: The transaction will revert with an 'insufficient funds for gas' error. The system logs this in MongoDB, alerts the admin dashboard, and returns a clean 503 error to the voter instructing them to retry in a few moments while the admin tops up the relay wallet.

C. Gasless Voting & Nonce Management

Examiner: What is a nonce in Ethereum?

Student: An account nonce is a counter indicating the number of transactions sent from a given account address. Nonces prevent double-spending and transaction replay attacks.

Examiner: How does your code handle nonces when 50 voters vote at once?

Student: We use two mechanisms in `Voting/lib/relay.js`: (1) We wrap the base wallet with `new ethers.NonceManager(baseWallet)` which locally manages nonce increments, and (2) We run all transactions through `sendRelayTx()` with a serialized promise queue and a retry loop with linear backoff ($800\text{ms} * \text{attempt}$) up to 4 attempts if an unexpected nonce error occurs.

D. Privacy & Nullifier Cross-Examination

Examiner: Can the database admin know who I voted for?

Student: In MongoDB, the `voter` document records that the voter has voted, but does not store the `candidateId`. The candidate choice is only logged in `VoteActivity` along with a blinded `ballotHash`, without storing the voter's email or nullifier. The two collections cannot be linked. However, as noted in our paper's honest limitations, the backend server process handles the HTTP payload in memory for a fraction of a second during submission.

Examiner: What if an attacker steals the MongoDB database?

Student: They cannot tamper with the election result because the official tally is calculated by the Ethereum smart contract, not MongoDB. Furthermore, they cannot decrypt voter passwords or OTPs because they are stored as one-way bcrypt hashes.

E. Authentication & Biometrics Cross-Examination

Examiner: Why did you use AWS Rekognition instead of running face detection completely in the phone?

Student: We use a defense-in-depth model: on-device ML Kit performs high-speed 90ms frame evaluation to track face presence and guide the user through the 240-degree sensor rotation. But client-side code can be decompiled or manipulated. Therefore, the captured image must be verified by a secure cloud provider—AWS Rekognition—which validates face confidence, lighting, open eyes, and lack of glasses before issuing a signed biometric JWT.

Examiner: Can someone hold up a photograph of the voter to bypass your system?

Student: No. The Android app requires the user to rotate the phone continuously through 240 degrees of yaw tracked by `SensorManager`. A flat photograph held in front of the camera cannot provide the proper parallax and motion dynamics, and ML Kit immediately resets progress if another person or face enters the background.

F. Smart Contract & Governance Cross-Examination

Examiner: What is UUPS, and why did you use it over Transparent Proxy?

Student: UUPS stands for Universal Upgradeable Proxy Standard (EIP-1822). In UUPS, the upgrade logic is stored inside the implementation contract rather than the proxy. This makes UUPS much more gas-efficient because normal calls do not need an extra `delegatecall` check, and it allows us to enforce custom upgrade rules, such as our 2-of-3 guardian multi-signature gate in `_authorizeUpgrade()`.

Examiner: Explain the underflow guard in `castVoteRelayedV3`.

Student: Look at `Voting/contracts/VotingV3.sol` line 393: When a voter changes their vote, we decrement the old candidate's vote count: `require(candidates[_electionId][oldCandidateId].voteCount > 0, "VotingV3: underflow guard")` before executing `candidates[_electionId][oldCandidateId].voteCount--`. This guarantees that an arithmetic underflow can never corrupt contract state even if state desynchronization occurred.

PART 4 — CODE-BASED VIVA PREPARATION

FILES I MUST KNOW FOR VIVA

File Path	Primary Purpose	Key Functions / Structs	Expected Examiner Question
Voting/contracts/VotingV3.sol	Core UUPS smart contract governing election state, nullifiers, re-voting, and tallies.	<code>castVoteRelayedV3()</code> , <code>registerVoterByRelay()</code> , <code>_authorizeUpgrade()</code> , <code>voteChoice</code> mapping.	"Explain how re-voting works without double counting."
Voting/lib/relay.js	Gasless relayer engine that sponsors gas, manages nonces, and serializes transactions.	<code>sendRelayTx()</code> , <code>enqueueRelay()</code> , <code>relayCastVote()</code> , <code>logRelayTransaction()</code> .	"How do you prevent nonce collisions during concurrent voting?"
Voting/lib/voterIdentity.js	Voter privacy module deriving deterministic on-chain nullifiers from email.	<code>computeNullifierHash(email)</code> .	"Show me the formula used to calculate the nullifier."
Voting/lib/voteAllowance.js	Business rules defining voting limits across app and polling station channels.	<code>checkVoteAllowance()</code> , <code>MAX_APP_VOTES = 2</code> , <code>VOTE_CHANNEL</code> .	"Why do you allow 2 votes in the app but only 1 at a station?"
Voting/lib/voteLedger.js	Concurrency control performing atomic vote slot reservations in MongoDB.	<code>reserveVoteSlot()</code> , <code>acquireCastLock()</code> , <code>releaseVoteSlot()</code> .	"What happens if two requests arrive from the same voter simultaneously?"
Voting/lib/biometric.js	Biometric token management and AWS Rekognition client initialization.	<code>issueBiometricToken()</code> , <code>verifyBiometricToken()</code> .	"What is inside the biometric token and how long does it live?"
Voting/app/api/auth/verify-otp/route.js	Core authentication route validating OTP, biometric token, and dispatching vote.	<code>POST(req)</code> handler.	"Trace this route from receiving the HTTP body to returning the txHash."
app/.../PhoneRotationTracker.kt	Android sensor tracker monitoring 240-degree yaw rotation during liveness check.	<code>onSensorChanged()</code> , <code>targetDegrees = 240f</code> .	"How does the phone know the user rotated 240 degrees?"

FUNCTIONS I MUST KNOW

- `castVoteRelayedV3(bytes32 _electionId, uint256 _candidateId, bytes32 _voterNullifier, bytes32 _ballotSalt)`:**
Solidity: Checks if voter is registered, checks Voting phase, retrieves `voteChoice[nullifier][electionId]`. If 0 (first vote), increments candidate's `voteCount`. If > 0 (re-vote), decrements old candidate's `voteCount` and increments new candidate's `voteCount`. Stores `_candidateId + 1` in `voteChoice` and emits `VoteCastPrivate`.
- `computeNullifierHash(email)`:**
JavaScript: Cleans email to lowercase and trims whitespace, retrieves `SERVER_IDENTITY_SECRET` from environment, and returns `ethers.keccak256(toUtf8Bytes(`${cleanEmail}:${secret}`))`.
- `sendRelayTx(sendFn, maxAttempts = 4)`:**
JavaScript: Pushes transaction function into `enqueueRelay()` promise queue. Loops up to 4 times; on nonce error, waits with linear backoff (`800 * attempt` ms) and retries. Waits for transaction receipt before completing.

- `reserveVoteSlot(voterId, channel) :`
JavaScript: Executes atomic MongoDB `Voter.findOneAndUpdate()` ensuring `stationVoteFinal != true` and `votesCast < 2`. Increments `votesCast` by 1. Returns null if limit exceeded.
- `issueBiometricToken(nullifierHash) :`
JavaScript: Creates a 10-minute HS256 JWT payload containing `{ nullifierHash, authenticated: true, exp }` signed with `JWT_SECRET`.

TOP 20 CODE QUESTIONS EXAMINERS WILL ASK

1. **Where is the smart contract deployed?** Deployed on Ethereum Sepolia testnet at address specified in `NEXT_PUBLIC_CONTRACT_ADDRESS`.
2. **Show me where re-voting decrements the old candidate.** In `VotingV3.sol` lines 397-398: `candidates[_electionId][oldCandidateId].voteCount--; candidates[_electionId][_candidateId].voteCount++;`.
3. **Why does voteChoice store candidateld + 1?** In Solidity, default mapping values are 0. If candidate ID 0 received a vote, storing 0 would mean 'not voted'. Storing `candidateId + 1` allows 0 to represent 'unvoted'.
4. **How is the ballot salt generated?** In `relay.js` line 137: `ethers.hexlify(ethers.randomBytes(32))`.
5. **Where is the UUPS 2-of-3 guardian check?** In `VotingV3.sol` lines 197-213: `_authorizeUpgrade()` checks `p.approvalCount >= APPROVAL_THRESHOLD` (which is 2).
6. **What prevents a guardian from approving twice?** In `VotingV3.sol` line 231: `require(!p.approvedBy[msg.sender], "VotingV3: already approved")`.
7. **Show me how the OTP is verified.** In `verify-otp/route.js` line 101: `await bcrypt.compare(String(otp).trim(), record.otp)`.
8. **Where are OTP attempt limits enforced?** In `verify-otp/route.js` line 94: `if (record.attempts >= 3) return error`.
9. **Show me where the station override locks the voter.** In `voteLedger.js` line 22: `if (channel === VOTE_CHANNEL.STATION) update.$set = { stationVoteFinal: true }`.
10. **Where is the 240-degree sensor target defined?** In `PhoneRotationTracker.kt` line 17: `val targetDegrees: Float = 240f`.
11. **How does the Android app detect multiple faces?** In `FaceCaptureCamera.kt` via Google ML Kit: `if (faces.size > 1, liveness state resets and shows warning`.
12. **Show me the AWS Rekognition call.** In `api/biometric/verify/route.js` line 58: `await rekognition.send(new DetectFacesCommand({ Image: { Bytes: liveBuffer }, Attributes: ['ALL'] })))`.
13. **Where are voting hours enforced?** In `lib/votingWindow.js` and checked before issuing OTP or casting votes (default 07:00 to 18:00 IST).
14. **Where is gas spend logged?** In `relay.js` line 98: logged to MongoDB `RelayTransaction` model with `txHash`, `gasUsed`, and `gasCostEth`.
15. **How do you verify inclusion without revealing the vote?** Via `GET /api/audit/verify?txHash=...` which calls `provider.getTransactionReceipt(txHash)` and checks block confirmation.
16. **What prevents unauthorized election creation?** In `VotingV3.sol` line 252: `createElection` has the `onlyRelay` modifier.
17. **Where are candidate party symbols hosted?** Uploaded via `POST /api/uploads` to AWS S3 and served via HTTPS URLs.
18. **Show me the NonceManager initialization.** In `relay.js` line 70: `_relayWallet = new ethers.NonceManager(baseWallet)`.
19. **What happens if a voter inputs the wrong OTP 3 times?** The record in `EmailOTP` is locked, and the voter must request a completely new OTP.
20. **Where is the 190-test benchmark executed?** In Hardhat using `npm hardhat test`, executing test suites in `test/VotingV3.test.js` and `test/attacks.test.js`.

FINAL SECTION — QUICK REVISION & 5-MINUTE PROJECT PRESENTATION

TOP 15 QUESTIONS I MUST MASTER

1. What is BlockVote?

A gasless, decentralized electronic voting system combining Ethereum smart contracts with Android biometric liveness and bounded re-voting.

Counter-question: Why not use existing Web3 voting apps like Snapshot? (Snapshot uses off-chain IPFS signatures; BlockVote enforces on-chain state and execution without requiring voter wallets).

2. How does gasless voting work?

The voter signs in with email OTP and biometrics; a backend relay wallet signs and broadcasts the blockchain transaction, paying all gas fees.

Counter-question: Doesn't that make the relay a central point of failure? (The relay can delay transactions, but cannot forge votes because only registered nullifiers and verified slots are accepted on-chain).

3. What is a nullifier?

A deterministic hash: `keccak256(email + secret)` that identifies a voter on-chain without revealing their email or identity.

Counter-question: What happens if two voters have the same nullifier? (Impossible unless two people share the exact same email address, which is unique per voter).

4. How is duplicate voting prevented?

On-chain: `isRegisteredVoter` checks registration and `voteChoice` updates the tally. Off-chain: MongoDB `reserveVoteSlot` atomically limits voters to 2 casts.

Counter-question: Why allow 2 casts instead of 1? (To allow one change of mind, mitigating remote vote coercion).

5. What is the polling station override?

A supervised web kiosk activated by an admin cookie where an in-person vote irrevocably supersedes any previous remote app vote.

Counter-question: Why is station vote final? (Because an in-person supervised voting booth provides a physically coercion-free environment).

6. What is stored on-chain?

Nullifier registration mappings, candidate vote counts, election phase, and salted private ballot hashes.

Counter-question: Are candidate choices visible in transactions? (No, VotingV3 emits salted hashes `keccak256(e || c || n || s)`).

7. How does re-voting affect the candidate count on-chain?

In `castVoteRelayedV3`, the contract decrements the old candidate's vote count and increments the new candidate's count in the same transaction.

Counter-question: What if the old count is zero? (Protected by an explicit underflow guard: `require(voteCount > 0)`).

8. What smart contract architecture did you use?

UUPS (Universal Upgradeable Proxy Standard, EIP-1822) written in Solidity 0.8.22 using OpenZeppelin Contracts Upgradeable v5.6.

Counter-question: Who can upgrade the contract? (Only a 2-of-3 quorum of guardian hardware wallets).

9. How do you prevent nonce collisions in the relayer?

We serialize transactions using a JavaScript promise queue and manage account nonces with `ethers.NonceManager`, backed by 4 retry attempts.

Counter-question: What is your measured throughput? (~300 transactions/hour on Ethereum Sepolia, bounded by block slot time).

10. How does the biometric liveness check work?

The Android app tracks a 240-degree head rotation using gyroscope sensors and ML Kit; the backend verifies image quality and open eyes via AWS Rekognition.

Counter-question: Do you store the voter's face? (No, images are processed in memory and discarded; only quality attributes and transient JWTs are issued).

11. How much gas does a vote cast consume?

A first vote consumes 89,946 gas; voter registration consumes 55,697 gas; a re-vote consumes 77,478 gas.

Counter-question: How much does that cost in real money? (At 1.09 gwei on Sepolia, one voter costs ~0.000159 ETH or \$0.40 USD).

12. How was your system tested?

190 automated tests including 75 contract tests, 30 adversarial security scenarios (A1 to A8), and validation against 114 live Sepolia transactions.

Counter-question: Did you test real attacks? (Yes, the suite tests unauthorized upgrades, out-of-range candidate IDs, salt replay, and concurrent burst submissions).

13. What is the role of MongoDB if you have blockchain?

MongoDB acts as an operational cache for voter rosters, bcrypt OTP records, rate limiting, and atomic reservation locks before blockchain relay.

Counter-question: If MongoDB crashes, is the election lost? (No. All election state, candidate tallies, and nullifier registrations are permanently on Ethereum).

14. Can an election admin change the election result?

No. The admin cannot directly update candidate vote counts or modify the contract logic without 2-of-3 guardian approvals.

Counter-question: Can the admin add a candidate during voting? (No. The contract strictly requires `phase == Registration` to add candidates).

15. What are the main limitations of BlockVote?

The relay sees the plaintext candidate in HTTP transit before submitting, and liveness testing does not match against enrolled government ID photos.

Counter-question: How would you fix this in future work? (Implement client-side homomorphic encryption / zero-knowledge snarks and integrate DigiLocker API).

5-MINUTE PROJECT EXPLANATION (VERBAL SCRIPT)

(Speak this naturally and confidently to your examiner)

"Good morning/afternoon, Sir. My project is **BlockVote**, a Gasless, Coercion-Mitigating Decentralized E-Voting System.

The Problem: Traditional voting methods suffer from serious flaws: paper ballots are slow and prone to booth capturing, EVMs require physical presence, and standard online voting puts complete trust in a centralized database administrator who can silently alter vote tallies. While blockchain offers decentralized immutability, standard Web3 voting forces voters to install MetaMask, safeguard private keys, and pay gas fees in cryptocurrency—making it unusable for ordinary citizens.

Our Solution: BlockVote bridges this gap by combining an upgradeable Ethereum smart contract (`VotingV3`) with a gasless backend relay and native Android client. The voter never touches cryptocurrency: our platform relay wallet pays all gas fees while serializing transactions through a promise queue with `ethers.NonceManager` to prevent nonce collisions.

Privacy & Anti-Coercion: To guarantee voter privacy, voter email addresses are never written to the blockchain; instead, we derive a deterministic `keccak256` nullifier using a server-side secret. Furthermore, the contract emits blinded ballot commitments rather than candidate IDs. To defeat voter coercion at home, BlockVote implements *bounded re-voting*—allowing a remote voter to cast a ballot and change it once—and a *supervised polling station override*, where an in-person kiosk ballot is final and overrides remote votes.

Authentication & Architecture: Voters authenticate via email OTP and biometric facial liveness. On Android, we track a 240-degree head rotation using hardware gyroscope sensors and Google ML Kit, followed by AWS Rekognition cloud validation. Privileged administrator actions and smart contract upgrades are governed by a 2-of-3 guardian multi-signature threshold under the UUPS proxy standard.

My Contribution & Results: As lead architect, I implemented the `VotingV3.sol` smart contract, the gasless relay in `relay.js`, the nullifier derivation, atomic MongoDB concurrency ledgers, and the verification pipeline. We thoroughly evaluated the system with 190 automated tests (including 30 adversarial security attack scenarios) and benchmarked gas across 114 live transactions on Ethereum Sepolia, demonstrating that a ballot costs just 89,946 gas (approximately \$0.40 USD at median testnet gas prices).

BlockVote proves that we can achieve the mathematical immutability and auditability of blockchain without sacrificing user simplicity or voter privacy. Thank you, Sir. I am ready for your questions."